

Содержание

1	Общие причины различий производительности	2
1.1	Интерпретатор vs компилятор	2
1.2	Накладные расходы выполнения (runtime overhead)	3
1.3	Универсальность vs производительность	4
1.4	Работа с памятью и кеш процессора	5
1.5	Константные факторы и асимптотика	6
1.6	Амортизация и редкие дорогие операции	8
1.7	Библиотечные оптимизации	10

Общие причины различий производительности

Интерпретатор vs компилятор

C++

Программа на C++ проходит следующий путь:

note

1 код -> компилятор -> машинный код -> выполнение на CPU

- компилятор заранее анализирует код
- применяет оптимизации
- генерирует эффективный машинный код

В итоге после компиляции процессор выполняет машинный код, сгенерированный из программы.

Python

Программа на Python проходит другой путь:

note

1 код -> байткод -> интерпретатор -> выполнение на CPU

Более точно:

1. Код компилируется во внутренний байткод (это происходит при запуске)
2. Интерпретатор (уже скомпилированная программа на C) начинает выполнение
3. Он:
 - читает инструкции байткода
 - определяет, что нужно сделать
 - выполняет соответствующий участок своего C-кода
4. После этого переходит к следующей инструкции

В Python появляется дополнительный слой — интерпретатор (посредник). А это дополнительные накладные расходы на каждую операцию

Накладные расходы выполнения (runtime overhead)

Runtime overhead — это дополнительные накладные расходы, которые не относятся напрямую к логике алгоритма. Например, при выполнении операции:

```
python
1 x + y
```

1. прочитать инструкцию
2. определить тип `x` и тип `y`
3. найти метод сложения и вызвать его
4. создать новый объект

Всё это — дополнительная работа помимо самой операции сложения.

Вызов функции в Python

В Python вызов функции — это тяжёлая операция.

```
python
1 ht.put(key, value)
```

Что происходит:

1. найти объект `ht`, метод `put`
2. подготовить вызов и передать аргументы
3. создать новый стек вызова (`frame`)
4. перейти внутрь функции и выполнить код
5. вернуться обратно и уничтожить `frame`

Вызов функции в C++

Что происходит:

1. адрес функции известен заранее
2. аргументы передаются через регистры/стек
3. выполняется переход к функции
4. выполняется код
5. возврат

Универсальность vs производительность

Объекты в Python

В Python всё является объектами. Даже числа — это объекты

```
python  
1 x = 5
```

Это не просто число, а объект, который содержит значение, тип, служебную информацию, механизмы управления памятью.

В следствие любая операция в Python:

```
python  
1 x + y
```

включает работу с объектами, вызов методов, создание нового объекта.

В C++:

```
C++  
1 int x = 5;
```

Это просто 4 байта в памяти, без дополнительной логики.

Итог

Python даёт:

- удобство разработки
- динамическую типизацию
- универсальные структуры

Но за это приходится платить:

- дополнительная логика (определение типов, поиск соответствующего метода и вызов его, создание объектов, обновление счётчиков ссылок (управление памятью))
- работой с объектами
- overhead интерпретатора

Работа с памятью и кеш процессора

CPU cache

Кеш — это очень быстрая память внутри процессора

Есть:

- RAM — медленная, большая
- Cache — быстрая, маленькая

CPU загружает не один элемент, а кусок памяти (cache line), например, 64 байта подряд

Python

Список — это динамический массив **указателей на объекты**:

note

```
1 [ ptr ][ ptr ][ ptr ][ ptr ]
2   v     v     v     v
3  obj   obj   obj   obj
```

- в массиве лежат только ссылки
- сами объекты находятся в разных местах памяти

Каждый элемент требует отдельного обращения к памяти

Минусы: больше обращений к RAM, хуже работает кеш, медленнее доступ

Почему в Python так

1. Объекты могут быть разного размера
2. Один объект может использоваться в нескольких местах
3. Универсальная модель хранения

C++

note

```
1 [ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
```

Данные лежат подряд, нет дополнительных переходов. А значит CPU загружает сразу несколько элементов

Плюсы: меньше обращений к памяти, высокая скорость

Константные факторы и асимптотика

Асимптотическая сложность описывает, как время работы алгоритма растёт при увеличении размера входных данных. Например, если алгоритм имеет сложность $O(n)$, это означает, что при достаточно больших n время работы растёт примерно линейно относительно размера входа.

Однако асимптотика не показывает точное время выполнения программы. Она игнорирует постоянные множители и менее значимые слагаемые. Поэтому две реализации могут иметь одинаковую асимптотическую сложность, но сильно отличаться по реальному времени работы.

Например, рассмотрим две функции времени:

note

```
1 T1(n) = 5n
2 T2(n) = 80n
```

Обе функции имеют линейную сложность:

note

```
1 T1(n) = O(n)
2 T2(n) = O(n)
```

Но на практике вторая функция будет примерно в 16 раз медленнее первой, потому что на каждый элемент выполняется больше работы.

То же самое происходит с реализациями структур данных. Даже если две реализации выполняют одну и ту же операцию с одинаковой теоретической сложностью, реальная стоимость одной операции может отличаться. Например, одна реализация может для каждого элемента выполнять несколько простых действий, а другая — дополнительные проверки, вызовы функций, обращения к памяти, создание объектов или переходы между разными участками кода.

Условно это можно представить так:

note

```
1 Быстрая реализация:
2 одна операция = 5 простых действий
3
4 Медленная реализация:
5 одна операция = 40 простых действий + вызов функции + несколько обращений к памяти
```

С точки зрения асимптотики обе реализации могут оставаться одинаковыми. Например, если каждая из них выполняет n операций, то обе имеют сложность $O(n)$. Но на графике медленная реализация будет находиться значительно выше, потому что каждая отдельная операция имеет большую постоянную стоимость.

Если две реализации показывают почти линейный рост, это говорит о близкой асимптотической сложности.

Но если одна из них стабильно в несколько раз быстрее, это означает, что у неё меньше постоянные расходы на одну операцию.

Константные факторы могут зависеть от разных причин:

- количества элементарных действий внутри операции;
- числа вызовов функций;
- количества проверок условий;
- числа обращений к памяти;
- эффективности использования кэша процессора;
- необходимости создавать временные объекты;
- уровня оптимизации компилятора или библиотеки;
- особенностей языка программирования и runtime.

Таким образом, асимптотика показывает общий закон роста времени, но не объясняет полностью практическую производительность. В реальных benchmark-ах особенно важно учитывать константные факторы, потому что именно они часто определяют разницу между реализациями с одинаковой теоретической сложностью.

Если две реализации имеют одинаковую асимптотику, но одна работает быстрее, это не противоречит теории сложности. Это означает, что быстрая реализация выполняет меньше работы на каждом шаге или эффективнее использует ресурсы процессора и памяти.

Амортизация и редкие дорогие операции

В некоторых структурах данных не каждая отдельная операция имеет одинаковую стоимость. Обычно операция может выполняться быстро, но иногда возникает необходимость выполнить дополнительную дорогую работу: расширить внутренний массив, перестроить структуру, перераспределить элементы или очистить накопленные служебные данные.

На первый взгляд это может выглядеть как нарушение ожидаемой сложности. Например, если обычная операция выполняется за $O(1)$, но иногда внезапно требует $O(n)$, возникает вопрос: почему тогда говорят, что операция всё равно имеет амортизированную сложность $O(1)$?

Идея амортизации состоит в том, что дорогая операция происходит редко, а её стоимость как бы “распределяется” по большому числу дешёвых операций.

Простой пример — динамический массив.

Обычно добавление элемента в конец массива выполняется быстро:

note

```
1 append(x)  -> O(1)
```

Если в массиве есть свободное место, новый элемент просто записывается в следующую позицию. Но когда массив полностью заполнен, нужно:

1. создать новый массив большего размера;
2. скопировать туда все старые элементы;
3. добавить новый элемент;
4. заменить старый массив новым.

Такая операция уже стоит $O(n)$, потому что приходится копировать n элементов.

Однако расширение происходит не при каждом добавлении. Если при переполнении размер массива увеличивается, например, в два раза, то после одного дорогого расширения появляется много свободных ячеек для будущих быстрых вставок. Поэтому на длинной последовательности операций средняя стоимость одного `append` остаётся $O(1)$.

Условно это можно представить так:

note

```
1 дешёво, дешёво, дешёво, дорого, дешёво, дешёво, дешёво, дешёво, дешёво, дорого, ...
```

Отдельная операция может быть дорогой, но таких операций мало относительно общего числа операций.

Например, если выполнить много добавлений подряд, суммарная стоимость будет складываться из двух частей:

note

1 обычные вставки + редкие расширения

Обычные вставки дают линейную стоимость. Расширения тоже в сумме дают линейную стоимость, потому что при стратегии увеличения размера в два раза каждый элемент копируется ограниченное число раз. Поэтому вся последовательность из N добавлений работает за $O(N)$, а средняя стоимость одной операции равна $O(1)$.

Важно понимать различие между обычной и амортизированной оценкой:

note

1 Обычная оценка одной конкретной операции: иногда может быть $O(n)$

2

3 Амортизированная оценка: средняя стоимость операции на длинной последовательности – $O(1)$

То есть амортизированная сложность не утверждает, что каждая операция всегда быстрая. Она утверждает, что дорогие операции происходят достаточно редко, чтобы не ухудшать среднюю стоимость на большой серии операций.

Такие редкие дорогие операции встречаются во многих структурах данных:

- динамический массив — расширение внутреннего массива;
- хеш-таблица — увеличение таблицы и перераспределение элементов;
- очереди и буферы — перераспределение внутреннего хранилища;
- некоторые деревья — периодическая балансировка или перестроение.

На графиках benchmark-ов амортизация обычно проявляется так: общий рост времени остаётся близким к ожидаемой асимптотике, но отдельные точки могут быть немного выше из-за того, что на данном размере произошло расширение или перестроение структуры.

Поэтому при анализе результатов важно не путать единичный скачок времени с изменением асимптотики. Если дорогие операции происходят редко, а общий тренд остаётся линейным или близким к ожидаемому, то это нормальное поведение для амортизированной структуры данных.

Итог: амортизация объясняет, почему структура может иногда выполнять дорогую операцию, но при этом сохранять хорошую среднюю сложность на длинной последовательности операций.

Библиотечные оптимизации

Библиотечные реализации структур данных часто работают быстрее учебных или пользовательских реализаций, даже если используют ту же самую базовую идею и имеют такую же асимптотическую сложность.

Причина в том, что промышленная библиотечная реализация обычно оптимизируется не только с точки зрения алгоритма, но и с точки зрения практического выполнения на реальном компьютере.

Например, две реализации могут иметь одинаковую сложность операции:

note

```
1 insert -> O(1)
2 get    -> O(1)
3 delete -> O(1)
```

Но это не означает, что они выполняют одинаковое количество реальной работы. Одна реализация может быть простой и понятной, а другая — содержать множество низкоуровневых оптимизаций, которые уменьшают константные расходы.

Библиотечные реализации обычно учитывают:

1. layout памяти;
2. кэш-локальность;
3. количество обращений к памяти;
4. число условных переходов;
5. стратегию роста внутреннего хранилища;
6. обработку частых случаев;
7. особенности компилятора и процессора;
8. минимизацию лишних проверок и временных объектов.

Например, если структура данных часто обращается к соседним элементам, библиотечная реализация может хранить данные более компактно, чтобы процессор эффективнее использовал cache line. В результате уменьшается число cache miss, и программа работает быстрее, хотя асимптотическая сложность не меняется.

Также библиотечная реализация может отдельно оптимизировать наиболее частые сценарии. В реальных программах некоторые операции встречаются гораздо чаще других, поэтому их делают максимально быстрыми. Например, если структура часто используется для вставки и поиска, эти операции могут быть реализованы особенно аккуратно: с меньшим числом проверок, с более удачной стратегией обхода памяти и с более эффективной обработкой коллизий или конфликтных случаев.

Важную роль играет и политика роста внутреннего хранилища. Простая реализация может расширять структуру корректно, но не всегда оптимально с точки зрения производительности. Библиотечная реализация обычно

выбирает такие коэффициенты роста и такие моменты перестроения, чтобы уменьшить число дорогих операций и сохранить хорошую работу с памятью.

Условно различие можно представить так:

note

- 1 Учебная реализация:
- 2 главная цель – понятность, корректность, простая логика
- 3
- 4 Библиотечная реализация:
- 5 главная цель – корректность + высокая производительность + устойчивость на разных данных

Поэтому учебная реализация может быть полезной и правильной, но при этом проигрывать библиотечной по времени выполнения. Это не обязательно означает ошибку в алгоритме. Часто причина в том, что библиотечная версия лучше оптимизирована на уровне деталей.

Кроме того, библиотечные структуры данных обычно проходят длительное тестирование и используются в большом количестве реальных проектов. Из-за этого в них постепенно накапливаются оптимизации для разных случаев: маленьких и больших размеров данных, частых вставок, большого числа поисков, плохого распределения ключей, удаления элементов и других сценариев.

Важно понимать, что библиотечная оптимизация не отменяет асимптотику. Если алгоритм имеет плохую теоретическую сложность, низкоуровневые оптимизации не смогут полностью это исправить на больших данных. Но если две реализации имеют одинаковую или близкую асимптотику, то именно библиотечные оптимизации часто определяют, какая из них будет быстрее на практике.

Например:

note

- 1 $T1(n) = 10n$
- 2 $T2(n) = 40n$

Обе реализации имеют сложность $O(n)$, но первая будет быстрее за счет меньших постоянных расходов. Библиотечные реализации как раз часто стремятся уменьшить этот множитель перед n .

Итог: библиотечные реализации часто быстрее не потому, что у них обязательно лучше асимптотика, а потому что они лучше оптимизированы на уровне памяти, частых операций, константных факторов и особенностей реального оборудования.